

pandas & polars

Lecture 08

Dr. Colin Rundel

pandas

pandas

Python has no built-in tabular structure, a data frame is something a library provides. pandas (2008) is the established choice - it is built on NumPy and borrows heavily from R, but differs from R's data frames in a few important ways that are the subject of this lecture.

Three classes do all of the work,

- **Series** - a 1d typed array with an index (a column)
- **DataFrame** - a collection of **Series** sharing an index (a table)
- **Index** - the labels attached to rows and columns

```
1 import pandas as pd
2 pd.__version__
```

py

```
'3.0.5'
```

pandas 3.0 (released early 2026) changed some long standing defaults - strings have their own dtype and copy-on-write

Series

A *Series* is a 1d array of one dtype together with an *index* of labels - the closest thing in R is a named vector. The difference is that a Series always has an index (integer positions by default) and the index participates in lookup, alignment, and joins rather than being decoration.

```
1 s = pd.Series([4, 2, 1, 3])
2 s
```

py

```
0    4
1    2
2    1
3    3
dtype: int64
```

```
1 s.index
```

py

```
RangeIndex(start=0, stop=4, step=1)
```

```
1 t = pd.Series(
2     [4, 2, 1, 3],
3     index=["a", "b", "c", "d"]
4 )
5 t
```

py

```
a    4
b    2
c    1
d    3
dtype: int64
```

dtypes

pandas dtypes come from NumPy (`int64`, `float64`, `bool`) plus its own extension dtypes (`str`, `category`, `datetime64`, ...). Where R coerces mixed values to a common type, pandas falls back to `object` - a column of arbitrary Python objects that behaves like a list rather than a vector, and is slow.

```
1 pd.Series([1, 2, 3]).dtype
```

py

```
dtype('int64')
```

```
1 pd.Series([1.5, 2]).dtype
```

py

```
dtype('float64')
```

```
1 pd.Series([True, False]).dtype
```

py

```
dtype('bool')
```

```
1 pd.Series(["a", "b"]).dtype
```

py

```
<StringDtype(na_value=nan)>
```

```
1 pd.Series([1, "a", True]).dtype
```

py

```
dtype('O')
```

```
1 pd.Series([1, "a", True])
```

py

```
0      1
1      a
2     True
dtype: object
```

```
1 pd.Series(["1", "2"]).astype("int64")
```

py

```
0      1
1      2
dtype: int64
```

Labels vs positions

A pandas index can hold integers, so `[]` on a Series would be ambiguous - it uses labels (`t[1]` is a `KeyError`), positions go through `.iloc` and labels through `.loc`. Label slices include their endpoint, position slices do not.

```
1 t["b"]
```

py

```
np.int64(2)
```

```
1 t.loc["b":"c"]
```

py

```
b    2
c    1
dtype: int64
```

```
1 t[t > 2]
```

py

```
a    4
d    3
dtype: int64
```

```
1 t.iloc[1]
```

py

```
np.int64(2)
```

```
1 t.iloc[1:3]
```

py

```
b    2
c    1
dtype: int64
```

```
1 u = pd.Series([4, 2, 1, 3],
2                 index=[3, 2, 1, 0])
3 u[1]
```

py

```
np.int64(1)
```

```
1 u.iloc[1]
```

py

```
np.int64(2)
```

Index alignment

The index is not just for lookup, arithmetic between Series aligns on labels rather than positions. The result spans the union of both indexes with **NaN** for labels missing on one side, and assigning a Series to a column aligns the same way.

```
1 m = pd.Series([1, 2, 3], index=["a", "b", "c"])
2 n = pd.Series([10, 20, 30], index=["c", "b", "a"])
3 d = pd.DataFrame({"x": [1, 2, 3]}, index=["a", "b", "c"])
```

```
1 m + n
```

```
a    31
b    22
c    13
dtype: int64
```

```
1 m + n[["a", "c"]]
```

```
a    31.0
b     NaN
c    13.0
dtype: float64
```

```
1 d["y"] = n[["a", "c"]]
2 d
```

	x	y
a	1	30.0
b	2	NaN
c	3	10.0

DataFrame

A `DataFrame` is a dict-like collection of Series sharing a row index, plus an index of column names - R's list of vectors with `names` and `row.names`. The difference is emphasis: R treats row names as an afterthought (tibbles drop them), the pandas index is central.

```
1 penguins = pd.read_csv("data/penguins.csv")
2 penguins
```

	species	island	bill_length_mm	...	body_mass_g	sex	year
0	Adelie	Torgersen	39.1	...	3750.0	male	2007
1	Adelie	Torgersen	39.5	...	3800.0	female	2007
..	...	...	...	...	...	...	...
342	Chinstrap	Dream	50.8	...	4100.0	male	2009
343	Chinstrap	Dream	50.2	...	3775.0	female	2009

[344 rows x 8 columns]

```
1 penguins.index
```

RangeIndex(start=0, stop=344, step=1)

```
1 penguins.columns
```

```
Index(['species', 'island', 'bill_length_mm', 'bill_depth_mm',  
      'flipper_length_mm', 'body_mass_g', 'sex', 'year'],  
      dtype='str')
```

Column types

Types are guessed from the file - there is no factor by default (`category` is opt in) and the two integer columns with missing values were upcast to `float64`. Asking for the `numpy_nullable` backend gives pandas' extension dtypes instead, which keep integers as integers (more on these shortly).

```
1 penguins.dtypes
```

py

```
species          str
island           str
bill_length_mm   float64
bill_depth_mm    float64
flipper_length_mm float64
body_mass_g      float64
sex              str
year             int64
dtype: object
```

```
1 pd.read_csv(
2     "data/penguins.csv",
3     dtype_backend="numpy_nullable"
4 ).dtypes
```

py

```
species          string
island           string
bill_length_mm   Float64
bill_depth_mm    Float64
flipper_length_mm Int64
body_mass_g      Int64
sex              string
year             Int64
dtype: object
```

Missing values

R has one **NA** that fits inside any vector type. NumPy has no missing value at all, so pandas adopted the floating point **NaN** as its sentinel - which means a missing value changes the dtype of a column: integers become **float64**, booleans become **object** holding **None**, and datetimes get their own **NaT**.

```
1 pd.Series([1, 2, None])
```

py

```
0    1.0
1    2.0
2    NaN
dtype: float64
```

```
1 pd.Series([True, False, None])
```

py

```
0    True
1   False
2    None
dtype: object
```

```
1 pd.Series(["a", None])
```

py

```
0    a
1   NaN
dtype: str
```

```
1 pd.Series(
2     [pd.Timestamp("2026-09-15"), None]
3 )
```

py

```
0    2026-09-15
1           NaT
dtype: datetime64[us]
```

Detecting missing values

`NaN` is not equal to anything, including itself, so `==` cannot find missing values - `isna()` (and `notna()`) recognize `NaN`, `None`, `NaT`, and `pd.NA` alike. `dropna()` and `fillna()` are the usual next steps.

```
1 s = pd.Series([1, 2, None])
2 s == np.nan
```

py

```
0    False
1    False
2    False
dtype: bool
```

```
1 s.isna()
```

py

```
0    False
1    False
2     True
dtype: bool
```

```
1 s.isna().sum()
```

py

```
np.int64(1)
```

```
1 s.dropna()
```

py

```
0    1.0
1    2.0
dtype: float64
```

```
1 s.fillna(0)
```

py

```
0    1.0
1    2.0
2    0.0
dtype: float64
```

Missing values in aggregations

Reductions skip missing values by default (`skipna=True`), the opposite of R's `na.rm = FALSE` - a mean silently ignores the gaps and the sum of an entirely missing column is 0. `count()` gives the number of non-missing values, `isna().sum()` the number missing.

```
1 mass = penguins["body_mass_g"]
```

py

```
1 mass.mean()
```

py

```
np.float64(4201.754385964912)
```

```
1 mass.mean(skipna=False)
```

py

```
np.float64(nan)
```

```
1 mass.count()
```

py

```
np.int64(342)
```

```
1 penguins.dropna().shape
```

py

```
(333, 8)
```

```
1 penguins.isna().sum()
```

py

```
species          0
island            0
bill_length_mm    2
bill_depth_mm     2
flipper_length_mm 2
body_mass_g       2
sex              11
year              0
dtype: int64
```

Nullable dtypes

The extension dtypes `Int64`, `Float64`, `boolean`, and `string` (capitalized, unlike NumPy's) have a proper missing value, `pd.NA`, which keeps the column's dtype and propagates through arithmetic and comparisons with three valued logic - `pd.NA == 1` is `<NA>` rather than `False`. They are opt in, via `dtype=`, `dtype_backend=` when reading, or `convert_dtypes()`.

```
1 t = pd.Series(  
2     [1, 2, None], dtype="Int64"  
3 )  
4 t
```

```
0      1  
1      2  
2    <NA>  
dtype: Int64
```

```
1 t > 1
```

```
0    False  
1     True  
2    <NA>  
dtype: boolean
```

```
1 pd.NA == 1
```

```
<NA>
```

```
1 pd.NA | True
```

```
True
```

```
1 pd.NA & True
```

```
<NA>
```

```
1 bool(pd.NA)
```

```
TypeError: boolean value of NA is ambiguous
```

So a pandas column can hold `NaN`, `None`, `NaT`, or `pd.NA` depending on its dtype - `isna()` treats all of them as missing,

Subsetting with []

`df[...]` is overloaded on the type of its argument - a string picks a column, a list of strings a set of columns, and a slice or boolean Series picks rows. There is no two argument `df[i, j]` form.

```
1 penguins["species"]
```

py

```
0      Adelie
1      Adelie
...
342  Chinstrap
343  Chinstrap
Name: species, Length: 344, dtype: str
```

```
1 penguins[["species", "island"]]
```

py

```
      species  island
0      Adelie  Torgersen
1      Adelie  Torgersen
..         ...      ...
342  Chinstrap    Dream
343  Chinstrap    Dream
```

[344 rows x 2 columns]

```
1 penguins[penguins["body_mass_g"] > 6000]
```

py

```
      species  island  bill_length_mm  ...  body_mass_g  sex  year
169  Gentoo  Biscoe         49.2  ...      6300.0  male  2007
185  Gentoo  Biscoe         59.6  ...      6050.0  male  2007
```

[2 rows x 8 columns]

.loc and .iloc

Two dimensional subsetting like R's `df[i, j]` goes through the `.loc` (labels) and `.iloc` (positions) accessors - each takes `[rows, cols]` where either can be a single value, a list, a slice, or a boolean mask. The same slice means different rows in the two.

```
1 penguins.loc[0:2, "species":"island"]
```

py

	species	island
0	Adelie	Torgersen
1	Adelie	Torgersen
2	Adelie	Torgersen

```
1 penguins.iloc[0:2, 0:2]
```

py

	species	island
0	Adelie	Torgersen
1	Adelie	Torgersen

```
1 penguins.loc[penguins["species"] == "Gentoo", ["species", "body_mass_g"]]
```

py

	species	body_mass_g
152	Gentoo	4500.0
153	Gentoo	5700.0
...	...	...
274	Gentoo	5200.0
275	Gentoo	5400.0

[124 rows x 2 columns]

Rows are Series too

Selecting a single row returns a Series, and a Series has one dtype - so a heterogeneous row is upcast to `object`. R keeps a single row as a data frame for exactly this reason. Use a list to get a one row DataFrame instead.

```
1 cols = ["species", "year"]
```

```
1 penguins.loc[0, cols]
```

```
species    Adelie
year        2007
Name: 0, dtype: object
```

```
1 r = penguins.loc[[0], cols]
2 r
```

```
species  year
0  Adelie  2007
```

```
1 r.dtypes
```

```
species    str
year       int64
dtype: object
```

Mutation

DataFrames are mutable Python objects - `df["z"] = ...` changes the frame in place, and any other name bound to the same object sees the change (reference semantics, Lecture 06). This is unlike R's copy-on-modify, where the same code would leave `d` untouched. `copy()` makes an independent frame, and methods such as `assign()` return a new one.

```
1 d = pd.DataFrame({"x": [1, 2, 3]})
2 d2 = d
3 d2["y"] = d2["x"] * 2
4 d
```

py

	x	y
0	1	2
1	2	4
2	3	6

```
1 d3 = d.copy()
2 d3["z"] = 1
3 d.columns
```

py

Index(['x', 'y'], dtype='str')

pandas 3.0 turned on copy-on-write - subsets and `.copy()` are always independent of their source, so aliasing a name is

Referring to columns

dplyr's `filter(df, x > 1)` works because R captures the unevaluated expression. Python evaluates arguments eagerly and a bare `x` is a `NameError`, so pandas offers several workarounds, none of which are quite as convenient.

```
1 pg = penguins[["species", "island", "body_mass_g"]]
```

py

Explicit - repeat the frame's name,

```
1 pg[pg["body_mass_g"] > 6000]
```

py

	species	island	body_mass_g
169	Gentoo	Biscoe	6300.0
185	Gentoo	Biscoe	6050.0

String - a mini language parsed by pandas,

```
1 pg.query("body_mass_g > 6000")
```

py

	species	island	body_mass_g
169	Gentoo	Biscoe	6300.0
185	Gentoo	Biscoe	6050.0

Callable - a function receiving the frame,

```
1 pg[lambda d: d["body_mass_g"] > 6000]
```

py

	species	island	body_mass_g
169	Gentoo	Biscoe	6300.0
185	Gentoo	Biscoe	6050.0

Expression - `pd.col()`, new in pandas 3.0,

```
1 pg[pd.col("body_mass_g") > 6000]
```

py

	species	island	body_mass_g
169	Gentoo	Biscoe	6300.0
185	Gentoo	Biscoe	6050.0

Method chaining

There is no pipe, but since nearly every method returns a new DataFrame the equivalent is a chain of methods, wrapped in parentheses so it can span lines. Callables and `pd.col()` are what let a step refer to columns of the intermediate result.

```
1 (penguins
2   .query("species == 'Gentoo'")
3   .assign(kg = pd.col("body_mass_g") / 1000)
4   .sort_values("kg", ascending=False)
5   [["species", "island", "kg"]]
6   .head(3)
7 )
```

	species	island	kg
169	Gentoo	Biscoe	6.30
185	Gentoo	Biscoe	6.05
229	Gentoo	Biscoe	6.00

groupby

Split-apply-combine works the same way - `groupby()` returns a lazy `DataFrameGroupBy` and nothing is computed until an aggregation is applied. Unlike `dplyr` the group keys become the *index* of the result (`as_index=False` keeps them as columns, like `.by`), and aggregations are given as `(column, function)` pairs.

```
1 g = penguins.groupby("species")
```

```
1 type(g)
```

```
<class 'pandas.api.typing.DataFrameGroupBy'>
```

```
1 g["body_mass_g"].mean()
```

```
species
Adelie      3700.662252
Chinstrap   3733.088235
Gentoo      5076.016260
Name: body_mass_g, dtype: float64
```

```
1 (penguins
2   .groupby("species", as_index=False)
3   .agg(
4       n = ("species", "size"),
5       mass = ("body_mass_g", "mean")
6   )
7 )
```

	species	n	mass
0	Adelie	152	3700.662252
1	Chinstrap	68	3733.088235
2	Gentoo	124	5076.016260

Grouped transforms

`transform()` returns a result the same length as its input, broadcasting each group's value back over that group's rows - the equivalent of `mutate()` with `.by`. Since the result shares the frame's index it can be used directly in arithmetic or assignment.

```
1 avg = g["body_mass_g"].transform("mean")
2 avg.head(3)
```

```
0    3700.662252
1    3700.662252
2    3700.662252
Name: body_mass_g, dtype: float64
```

```
1 penguins.assign(rel = pd.col("body_mass_g") / avg)[["species", "rel"]].head(3)
```

```
   species      rel
0  Adelie  1.013332
1  Adelie  1.026843
2  Adelie  0.878221
```

MultiIndex

Group by two keys and the result has a hierarchical `MultiIndex`, aggregate a column two ways and the columns get one too. There is no R analog (the tidyverse keeps keys as ordinary columns) and most code is simpler after a `reset_index()` back to a flat table.

```
1 (penguins
2   .groupby(["species", "island"])
3   ["body_mass_g"].mean()
4   )
```

species	island	
Adelie	Biscoe	3709.659091
	Dream	3688.392857
	Torgersen	3706.372549
Chinstrap	Dream	3733.088235
Gentoo	Biscoe	5076.016260

Name: body_mass_g, dtype: float64

```
1 (penguins
2   .groupby("species")
3   .agg({
4       "body_mass_g": ["mean", "max"]
5   })
6   )
```

	body_mass_g	
	mean	max
species		
Adelie	3700.662252	4775.0
Chinstrap	3733.088235	4800.0
Gentoo	5076.016260	6300.0

`.index` on the left result and `.columns` on the right are both `MultiIndex` objects - indexing into them uses tuples,

Exercise 1

Using pandas and `data/penguins.csv`,

1. Select the rows for Gentoo penguins with a body mass above 5000 g, keeping only the `species`, `island`, and `body_mass_g` columns.
2. Add a column `body_mass_kg` containing the body mass in kilograms.
3. Calculate the mean bill length and the number of penguins for each combination of species and island - the result should be a flat table with `species` and `island` as columns, not an index.



polars

polars (2020) is a newer data frame library written in Rust on top of Apache Arrow's memory layout, with bindings for Python, R, and JavaScript. Rather than extend NumPy conventions it was designed from scratch around a few ideas,

- No index - rows are only ever positions
- A strict schema - every column's dtype is known and never silently changes
- Expressions - computations on columns are objects, built up and then evaluated in a context
- Lazy evaluation - queries are planned and optimized before running, on all available cores

```
1 import polars as pl
2 pl.__version__
```

py

```
'1.44.2'
```

Series

A polars Series is a name, a dtype, and values - nothing else. With no index there is nothing to align on, so arithmetic is positional and lengths must match (or one side has length one). This is R's rule without recycling.

```
1 pl.Series("x", [4, 2, 1, 3])
```

py

shape: (4,)

x
4
2
1
3

```
1 a = pl.Series([1, 2, 3])
2 b = pl.Series([10, 20, 30])
3 a + b
```

py

shape: (3,)

11
22
33

DataFrame

The print shows the shape and every column's dtype, much like a tibble. Note that the integer columns stayed integers despite the missing values, and that "NA" had to be declared as a missing value marker - polars does not guess.

```
1 penguins = pl.read_csv("data/penguins.csv", null_values="NA")
2 penguins
```

shape: (344, 8)

species	island	bill_length_mm	...	body_mass_g	sex	year
str	str	f64	...	i64	str	i64
"Adelie"	"Torgersen"	39.1	...	3750	"male"	2007
"Adelie"	"Torgersen"	39.5	...	3800	"female"	2007
"Adelie"	"Torgersen"	40.3	...	3250	"female"	2007
...	...	...	...	...	...	...
"Chinstrap"	"Dream"	49.6	...	3775	"male"	2009
"Chinstrap"	"Dream"	50.8	...	4100	"male"	2009
"Chinstrap"	"Dream"	50.2	...	3775	"female"	2009

Subsetting

Without an index there is no `.loc` / `.iloc` distinction - `df[...]` takes column names or row positions and a slice is just a slice. This is fine for a quick look, but the idiomatic way to subset is with expressions in `filter()` and `select()`.

```
1 penguins["species"]
```

py

shape: (344,)

species
str
"Adelie"
"Adelie"
"Adelie"
...
"Chinstrap"
"Chinstrap"
"Chinstrap"

```
1 penguins[0:3, ["species", "island"]]
```

py

shape: (3, 2)

species	island
str	str
"Adelie"	"Torgersen"
"Adelie"	"Torgersen"
"Adelie"	"Torgersen"

```
1 penguins[0, "species"]
```

py

'Adelie'

Missing values

Missing data is `null` in every dtype (like R's `NA`) and `NaN` is just a floating point value, distinct from missing. Aggregations skip `null` by default and propagate `NaN`.

```
1 s = pl.Series([1.0, None, np.nan])
2 s
```

py

shape: (3,)

	f64
1.0	
null	
NaN	

```
1 s.is_null().to_list()
```

py

[False, True, False]

```
1 s.is_nan().to_list()
```

py

[False, None, True]

```
1 s.mean()
```

py

nan

```
1 s.fill_nan(None).mean()
```

py

1.0

Expressions

`pl.col("x")` does not return a column, it returns an *expression* - a description of a computation that is not attached to any DataFrame and has not been evaluated. Expressions compose with operators and methods and are only evaluated when handed to a context. This is polars' answer to data masking - build an object that represents the code rather than capturing it.

```
1 mass_kg = pl.col("body_mass_g") / 1000
2 mass_kg
```

```
[(col("body_mass_g")) / (dyn int: 1000)]
```

```
1 type(mass_kg)
```

```
<class 'polars.expr.expr.Expr'>
```

```
1 pl.col("body_mass_g").mean().alias("avg_mass")
```

```
col("body_mass_g").mean().alias("avg_mass")
```

Contexts

A context is a DataFrame method that evaluates expressions against its columns, each corresponding to a dplyr verb,

Context	Result	dplyr
<code>select()</code>	a new frame containing only these expressions	<code>select(), summarize()</code>
<code>with_columns()</code>	the frame with these expressions added	<code>mutate()</code>
<code>filter()</code>	the rows where these expressions are true	<code>filter()</code>
<code>group_by().agg()</code>	one row per group of these expressions	<code>group_by() > summarize()</code>

```
1 penguins.select(  
2   "species", mass_kg.alias("mass_kg")  
3 ).head(2)
```

shape: (2, 2)

species	mass_kg
str	f64
"Adelie"	3.75
"Adelie"	3.8

```
1 penguins.filter(  
2   pl.col("body_mass_g") > 6000  
3 ).select("species", "body_mass_g")
```

shape: (2, 2)

species	body_mass_g
str	i64
"Gentoo"	6300
"Gentoo"	6050

Expression expansion

One expression can expand to many columns - by listing names, by dtype, or with selectors from `polars.selectors`. This is `across()` with tidy selection, available in every context.

```
1 import polars.selectors as cs
```

py

```
1 penguins.select(cs.ends_with("_mm").mean().name.suffix("_mean"))
```

py

shape: (1, 3)

bill_length_mm_mean	bill_depth_mm_mean	flipper_length_mm_mean
f64	f64	f64
43.92193	17.15117	200.915205

```
1 penguins.select(pl.col(pl.String).n_unique())
```

py

shape: (1, 3)

species	island	sex
u32	u32	u32
3	3	3

group_by

`group_by()` followed by `agg()` evaluates each expression once per group and the keys come back as ordinary columns (like `.by`), in no guaranteed order. Since expressions are values the same one can be reused across contexts, and `.over()` turns an aggregation into a window computation for a grouped mutate.

```
1 mass = pl.col("body_mass_g")
2 penguins.group_by("species").agg(
3     n = pl.len(), avg = mass.mean()
4 ).sort("species")
```

py

shape: (3, 3)

species	n	avg
str	u32	f64
"Adelie"	152	3700.662252
"Chinstrap"	68	3733.088235
"Gentoo"	124	5076.01626

```
1 avg = mass.mean().over("species")
2 (penguins
3     .with_columns(rel = mass / avg)
4     .select("species", "rel")
5     .head(2)
6 )
```

py

shape: (2, 2)

species	rel
str	f64
"Adelie"	1.013332
"Adelie"	1.026843

Lazy evaluation

A `LazyFrame` has the same API as a `DataFrame` but records the query instead of running it. The `scan_*()` readers start lazy, `.lazy()` converts an existing frame, and nothing is computed until `.collect()` is called.

```
1 q = (  
2     pl.scan_parquet("data/flights.parquet")  
3     .filter(pl.col("dest") == "LAX")  
4     .group_by("carrier")  
5     .agg(pl.len(), pl.col("dep_delay").mean())  
6     .sort("len", descending=True)  
7 )  
8 type(q)
```

py

```
<class 'polars.lazyframe.frame.LazyFrame'>
```

Recording the whole query before running it is what makes optimization possible - the same idea underlies SQL databases and dbplyr, which we will see later in the course.

Query plan

Before running, the plan is optimized - here only 3 of the 19 columns are read from the parquet file and the filter is applied during the scan (projection and predicate *pushdown*).

```
1 print( q.explain() )
```

py

```
SORT BY [descending: [true]] [col("len")]
  AGGREGATE[maintain_order: false]
    [len(), col("dep_delay").mean()] BY [col("carrier")]
  FROM
    simple π 2/2 ["carrier", "dep_delay"]
      Parquet SCAN [data/flights.parquet]
      PROJECT 3/19 COLUMNS
      SELECTION: col("dest") == "LAX"
      ESTIMATED ROWS: 336776
```

Collecting

`collect()` runs the optimized plan and returns an ordinary DataFrame. Since the plan is known up front it can also be executed in a streaming fashion, in batches, for data larger than memory (`collect(engine="streaming")`).

```
1 q.collect()
```

py

shape: (5, 3)

carrier	len	dep_delay
str	u32	f64
"UA"	5823	10.757738
"AA"	3582	9.168119
"VX"	2580	10.56189
"DL"	2501	5.668539
"B6"	1688	8.994652

Speed

Both libraries are compiled underneath, but polars is multithreaded and its optimizer can avoid work entirely. The gap is largest for parsing text and for pipelines where pushdown reduces the data read.

```
1 import timeit
2 def time(f): return round( min(timeit.repeat(f, number=1, repeat=5)), 4 )
```

```
1 time(lambda: pd.read_csv("data/flights.csv"))
```

0.224

```
1 time(lambda: pl.read_csv("data/flights.csv", null_values="NA"))
```

0.01

```
1 flights_pd = pd.read_parquet("data/flights.parquet")
2 time(lambda: flights_pd.groupby("carrier")["dep_delay"].mean())
```

0.0033

```
1 time(lambda: q.collect())
```

0.0017

Interop

All three implementations can exchange data through Apache Arrow - polars and pandas convert directly, and R's arrow and reticulate packages move tables in and out of Python. The dtype names differ but the underlying data does not. For anything on disk, parquet is the shared format (Lecture 07).

```
1 penguins.to_pandas().dtypes
```

py

```
species          str
island           str
bill_length_mm   float64
bill_depth_mm    float64
flipper_length_mm float64
body_mass_g      float64
sex              str
year             int64
dtype: object
```

```
1 penguins.to_arrow().schema
```

py

```
species: large_string
island: large_string
bill_length_mm: double
bill_depth_mm: double
flipper_length_mm: int64
body_mass_g: int64
sex: large_string
year: int64
```

Exercise 2

Using polars and `data/penguins.csv`,

1. Repeat the three tasks from Exercise 1 - select the Gentoo penguins above 5000 g keeping `species`, `island`, and `body_mass_g`, add a `body_mass_kg` column, and calculate the mean bill length and count for each species and island.
2. Rewrite part 3 to start from `pl.scan_csv()` and look at the output of `explain()` before collecting - how many of the 8 columns does polars read?

Summary

Three data frames

Concept	R / tibble	pandas	polars
column	atomic vector	<code>Series</code> (values + index)	<code>Series</code> (values only)
row labels	<code>row.names</code> , rarely used	<code>Index</code> , central	none
alignment	by position, with recycling	by index label	by position, lengths must match
mixed types	coerced to a common type	<code>object</code> dtype	error
missing values	<code>NA</code> in any type	<code>NaN</code> / <code>None</code> / <code>NaT</code> / <code>pd.NA</code> by dtype	<code>null</code> in any type
column references	data masking	strings, callables, <code>pd.col()</code>	<code>pl.col()</code> expressions
mutation	copy-on-modify	in place, copy-on-write for subsets	immutable
grouped result	keys as columns	keys as (Multi)Index	keys as columns
laziness	none (dbplyr, dtplyr)	none	<code>LazyFrame</code> + optimizer
engine	C, single threaded	NumPy, single threaded	Rust + Arrow, multithreaded

Verbs

dplyr	pandas	polars
<code>filter(x > 1)</code>	<code>query("x > 1"), [pd.col("x") > 1]</code>	<code>filter(pl.col("x") > 1)</code>
<code>select(x, y)</code>	<code>[["x", "y"]], filter(regex=)</code>	<code>select("x", "y"), select(cs....())</code>
<code>mutate(z = x * 2)</code>	<code>assign(z = pd.col("x") * 2)</code>	<code>with_columns(z = pl.col("x") * 2)</code>
<code>arrange(desc(x))</code>	<code>sort_values("x", ascending=False)</code>	<code>sort("x", descending=True)</code>
<code>summarize(m = mean(x))</code>	<code>agg(m = ("x", "mean"))</code>	<code>select(m = pl.col("x").mean())</code>
<code>group_by(g)</code>	<code>groupby("g", as_index=False)</code>	<code>group_by("g")</code>
<code>mutate(m = mean(x), .by = g)</code>	<code>groupby("g")["x"].transform("mean")</code>	<code>pl.col("x").mean().over("g")</code>
<code>across(where(is.numeric), f)</code>	<code>select_dtypes("number").apply(f)</code>	<code>cs.numeric().f()</code>
<code>rename(new = old)</code>	<code>rename(columns={"old": "new"})</code>	<code>rename({"old": "new"})</code>
<code>left_join(y, by = "k")</code>	<code>merge(y, on="k", how="left")</code>	<code>join(y, on="k", how="left")</code>
<code>pivot_longer() / pivot_wider()</code>	<code>melt() / pivot()</code>	<code>unpivot() / pivot()</code>

Takeaways

- A pandas Series is a typed array plus an index, and the index is what makes pandas different from R - lookup is by label, arithmetic and assignment align on labels, and group keys end up in the index. `.loc` and `.iloc` exist because `[]` cannot tell labels from positions.
- pandas dtypes are messier than R's - mixed data becomes `object`, and missing values are `NaN`, `None`, `NaT`, or `pd.NA` depending on the dtype, with `NaN` silently turning integer columns into floats. Use `isna()`, never `==`, and remember that aggregations skip missing values by default.
- Python has no data masking, so pandas refers to columns with strings, callables, or `pd.col()` and chains methods instead of piping. DataFrames are mutable, so aliases matter.
- polars drops the index and adds a strict schema, `null` for every dtype, expressions evaluated inside contexts, and a lazy mode with a query optimizer - much of which will look familiar again when we get to SQL and DuckDB.